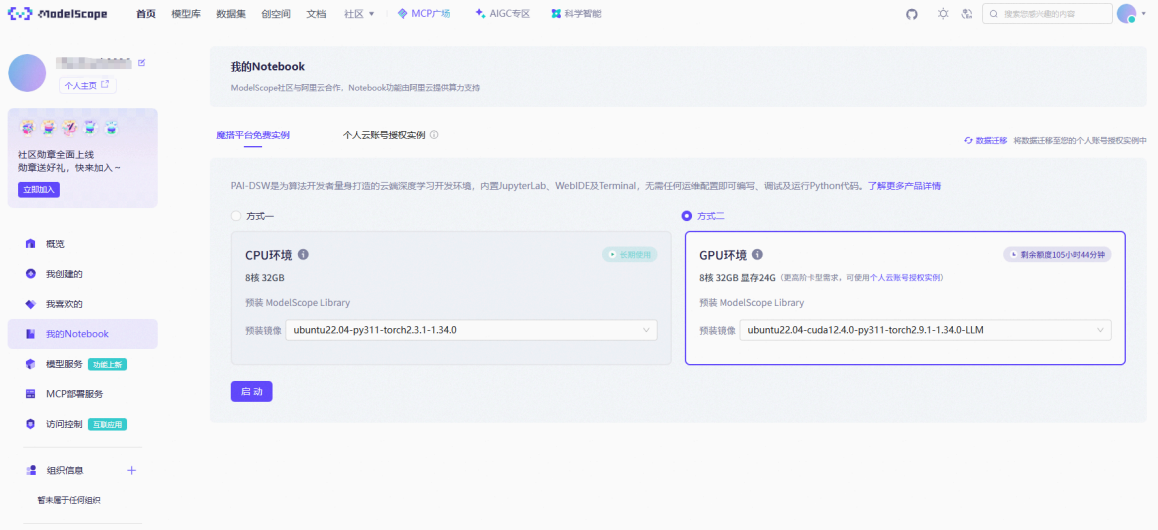
微调 LLM 教程

这份教程源于李宏毅老师《生成式人工智能》 hw5 LLM 微调作业，前人做了汉化和本地化([尝试微调 LLM：让它会写唐诗])，并介绍了基本的SFT代码流程。不过，由于作业时间比较早(2024年发布)，距离现在也有段时间，因此有的代码或者框架可能会比较落后，但不影响SFT的整个流程与结果。

因此，教程的前半部分还是在基于唐诗生成的微调任务介绍来讲解代码流程，对应的代码为 `LLM_fine-tuning.ipynb`；后半部分介绍我们的任务，即基于MDD-5k数据集微调LLM模型，这部分大家可以根据前面给的代码进行修改运行，也可以去尝试使用 LlamaFactory 等框架进行微调。

运行这些程序需要一定的算力，本地运行可能时间会较为漫长，可以使用免费的魔搭([ModelScope 魔搭社区]) **GPU环境**运行。AutoDL、Kaggle、Colab等平台也可以。



目录

微调唐诗部分：

- [环境配置与库安装]
- [导入库]
- [下载微调所需的数据集]
- [定义辅助函数]
  - [数据预处理函数]
  - [模型评估函数]
- [下载模型并在微调前进行推理]
  - [选择预训练模型]
  - [微调前的推理]
    - [加载模型]
    - [初始表现]
- [设置用于微调的超参数]
- [开始微调]

- [测试微调后的模型]
  - [选取要加载的微调后的模型]
  - [释放显存]
  - [加载模型和分词器]
  - [生成测试结果]
- [微调前后的对比]

微调咨询师部分：

- [MDD5k 数据集介绍]
- [任务建议]
- [其他参考内容]

环境配置与库安装

首先，我们需要安装一些必要的库，以便简化微调过程。这里我们使用 [`uv`] 安装，当然也可以使用 `pip` 安装。

```
uv add bitsandbytes
uv add datasets
uv add transformers
uv add peft
uv add sentencepiece
uv add accelerate
uv add colorama
uv add fsspec
```

库说明：

- `bitsandbytes`：用于低精度计算，加速训练过程。
- `datasets`：用于加载和处理数据集。
- `transformers`：Hugging Face 提供的库，包含预训练的模型和 Tokenizer。
- `peft`：Parameter-Efficient Fine-Tuning，参数高效微调库。
- `sentencepiece`：用于处理分词。
- `accelerate`：用于加速训练过程。
- `colorama`：用于在终端中打印彩色文本。
- `fsspec`：文件系统规范库。

如果是使用本地电脑的话，你还需要确定你的电脑安装了符合版本要求的 CUDA （[超详细 CUDA 安装与卸载教程（图文教程）]），如果使用服务器的话一般不用考虑(通常会已经帮你安装好了)。

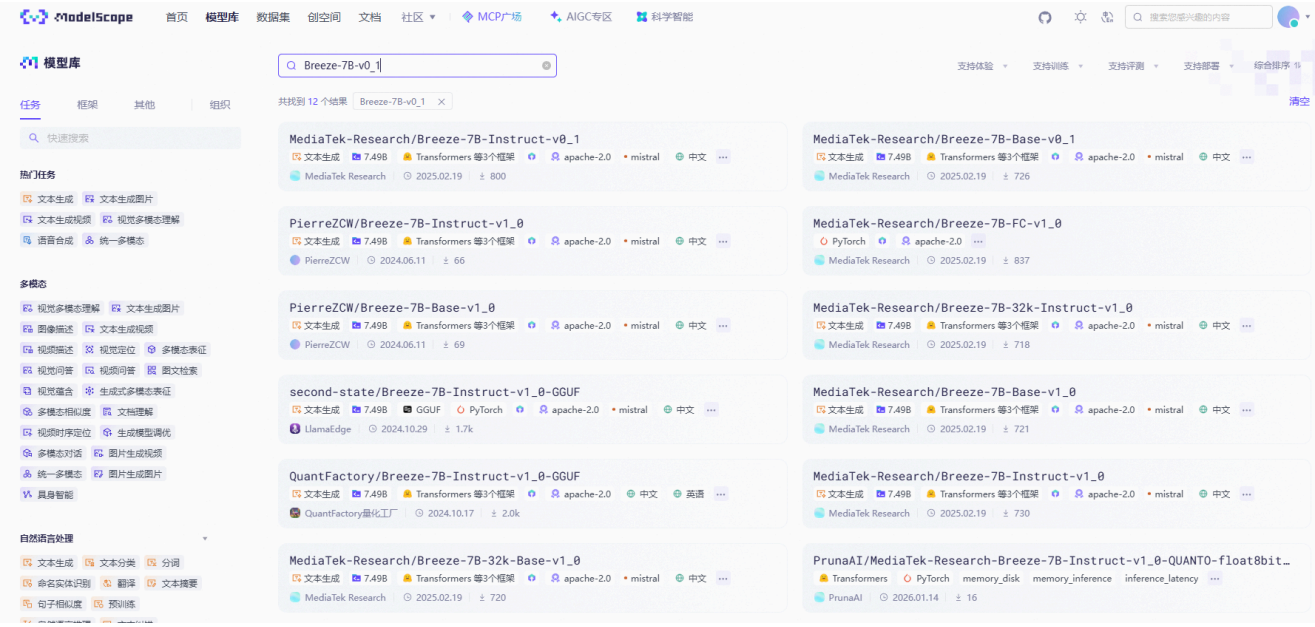
导入库

安装完库后，导入我们需要的模块。

```
import os
# 设置模型下载镜像（注意，需要在导入 transformers 等模块前进行设置才能起效）
os.environ['HF_ENDPOINT'] = 'https://hf-mirror.com'

import sys
import argparse
import json
import warnings
import logging
warnings.filterwarnings("ignore")

import torch
import torch.nn as nn
import bitsandbytes as bnb
from datasets import load_dataset, load_from_disk
import transformers
from peft import PeftModel
from colorama import Fore, Style

from tqdm import tqdm
from transformers import (
    AutoTokenizer,
    AutoConfig,
    AutoModelForCausalLM,
    BitsAndBytesConfig,
    GenerationConfig
)
from peft import (
    LoraConfig,
    get_peft_model,
    get_peft_model_state_dict,
```

```
    prepare_model_for_kbit_training
)
```

注意，由于 huggingface 网站在国内无法直接访问，因此需要通过[镜像方式]进行连接以下载模型库。另外也可以用国内的魔搭进行模型下载。



下载微调所需的数据集

我们将使用一个唐诗的数据集来微调模型，这个数据集来自于[全唐诗 - Github]。

```
git clone https://github.com/CheeEn-Yu/GenAI-Hw5.git
```

克隆完成后，数据集会存储在 `GenAI-Hw5` 目录下。

数据集内部信息是什么样的？

通常 SFT 的数据格式为 `json` ，会包含以下字段：

- **Instruction（指令）** ：Instruction 是**对模型的输入文本或问题进行进一步说明或约束的指令**。它可以是一种特定的格式或标记，用于告诉模型如何处理输入数据，或者提供一些额外的信息，以便模型更好地理解和生成输出
- **Input（输入）** ：Input 是模型实际接收的文本或数据。**它由 Prompt 和 Instruction 组成**，提供了完整的上下文和指导，用于生成输出。
- **Response（输出）** ：Response 是 LLMs 生成的输出文本或回答。**它是模型对 Input的理解和处理结果**。 Response 的内容取决于 LLMs 的能力和训练数据，它可以是文本、回答、建议、解释等形式。

  下载数据集后，会发现里面的 data 就是按照这样进行组织的：

```
{
  "instruction": "以下是一首唐詩的第一句話，請用你的知識判斷並完成整首詩。",
  "input": "秦川雄帝宅，函谷壯皇居。",
  "output": "綺殿千尋起，離宮百雉餘。連甍遙接漢，飛觀迥凌虛。雲日隱層闕，風煙出綺疎。"
},
```

其中，`output` 字段对应的就是上面介绍的 Response。

不过现在的SFT微调框架(例如 LlamaFactory 等) 通常有固定的数据格式 alpaca 和 sharegpt，因此如果想要用这些微调框架的话，建议将 data 转换成这样的格式。相关教程参考 [LLamaFactory:指令微调数据集格式与准备(Updating)]

定义辅助函数

在微调过程中，我们需要一些辅助函数来处理数据和评估模型。

数据预处理函数

```
def generate_training_data(data_point):
    """
    将输入和输出文本转换为模型可读取的 tokens。
    参数:
        data_point: 包含 "instruction"、"input" 和 "output" 字段的字典
    返回:
        包含模型输入 IDs、标签和注意力掩码的字典

    示例:
    - 如果你构建了一个字典 data_point_1，并包含字段 "instruction"、"input" 和 "output"，你可以像这样使用函数:
        generate_training_data(data_point_1)
    """
    # 构建完整的输入提示词
    prompt = f"""\
[INST] <<SYS>>
You are a helpful assistant and good at writing Tang poem. 你是一個樂於助人的助手且擅長寫唐詩。
<</SYS>>

{data_point["instruction"]}
{data_point["input"]}
[/INST]"""
```

```python
    # 计算用户提示词的 token 数量
    len_user_prompt_tokens = (
        len(
            tokenizer(
                prompt,
                truncation=True,
                max_length=CUTOFF_LEN + 1,
                padding="max_length",
            )["input_ids"]
        ) - 1
    )

    # 将完整的输入和输出转换为 tokens
    full_tokens = tokenizer(
        prompt + " " + data_point["output"] + "</s>",
        truncation=True,
        max_length=CUTOFF_LEN + 1,
        padding="max_length",
    )["input_ids"][:-1]

    return {
        "input_ids": full_tokens,
        "labels": [-100] * len_user_prompt_tokens +
full_tokens[len_user_prompt_tokens:],
        "attention_mask": [1] * len(full_tokens),
    }
```

函数解释:

- **目的**: 将原始数据转换为模型可以处理的输入格式。
- **处理步骤**:
  - 构建提示词 `prompt`，包括系统信息和用户的指令。
  - 使用 `tokenizer` 将 `prompt` 转换为 token，并计算其长度。
  - 将完整的输入（提示词和输出）转换为 tokens。
  - 构建 `labels`，对于提示词部分的 tokens，使用 `-100`（在计算损失时会被忽略），对于输出部分的 tokens，保留实际的 token ID。

为什么要这样做?

1. 提示词模板部分，[INST] 和 <<\SYS>>实际上是特殊token，标记着prompt中各个部分的构成，这是 llama2 用于区分用户输入的。[^2]

2. tokenizer 对文本进行分词、截断和填充，不同模型有不同的tokenizer，详见 [Stanford CS336 | Language Modeling from Scratch (2025)]
3. 损失掩码，在PyTorch的CrossEntropyLoss中，-100位置的token不参与损失计算，使得 NTP 时这部分不计入损失
4. 注意力掩码，指示哪些token是真实内容（1），哪些是填充token（0）

模型评估函数

```
def evaluate(instruction, generation_config, max_len,
input_text="", verbose=True):
    """
    获取模型在给定输入下的生成结果。

    参数:
        instruction: 描述任务的字符串
        generation_config: 模型生成配置
        max_len: 最大生成长度
        input_text: 输入文本，默认为空字符串
        verbose: 是否打印生成结果

    返回:
        模型生成的文本
    """
    # 构建完整的输入提示词
    prompt = f"""\
[INST] <<SYS>>
You are a helpful assistant and good at writing Tang poem. 你是一個樂於助人的助手且擅長寫唐詩。
<</SYS>>

{instruction}
{input_text}
[/INST]"""

    # 将提示词转换为模型所需的 token 格式
    inputs = tokenizer(prompt, return_tensors="pt")
    input_ids = inputs["input_ids"].cuda()

    # 使用模型生成回复
    generation_output = model.generate(
```

```
        input_ids=input_ids,
        generation_config=generation_config,
        return_dict_in_generate=True,
        output_scores=True,
        max_new_tokens=max_len,
    )

    # 解码并打印生成的回复
    for s in generation_output.sequences:
        output = tokenizer.decode(s)
        output = output.split("[/INST]")[1].replace("</s>",
"").replace("<s>", "").replace("Assistant:",
"").replace("Assistant", "").strip()
        if verbose:
            print(output)

    return output
```

函数解释:

- **目的**: 给定一个指令，使用模型生成对应的回复。
- **处理步骤**:
  - 构建提示词 `prompt`，包括系统信息和用户的指令。
  - 使用 `tokenizer` 将 `prompt` 转换为模型的输入格式。
  - 调用 `model.generate` 生成文本。
  - 对生成的序列进行解码，提取模型的输出部分。

下载模型并在微调前进行推理

选择预训练模型

我们将默认选择 MediaTek Breeze 7B 模型，因为 TAIDE 模型可能遇到下载次数过多无法访问。实际上换其他的开源大模型一样可以，并不影响学习。

```
""" 你可以（但不一定需要）更改 LLM 模型 """

model_name = "MediaTek-Research/Breeze-7B-Instruct-v0_1"

# model_name = "/content/TAIDE-LX-7B-Chat"
# 如果你想使用 TAIDE 模型，请先查看 TAIDE L Models Community License
Agreement
```

```
(https://drive.google.com/file/d/1FcUZjbUH6jr4xoCyAronN_slLgcdhEUd
/view)。
# 一旦使用，即表示你同意协议条款。
# !wget -O taide_7b.zip
"https://www.dropbox.com/scl/fi/harnetdwx2ttq1xt94rin/TAIDE-LX-7B-
Chat.zip?rlkey=yzyf5nxztw6farpwyyildx5s3&st=s22mz5ao&dl=0"
# !unzip taide_7b.zip
```

微调前的推理

先看看在未进行微调的情况下，模型的表现如何。

加载模型

会占用大概 5.2G 的显存。

```
cache_dir = "./cache"

# 进行模型量化
nf4_config = BitsAndBytesConfig(
   load_in_4bit=True,
   bnb_4bit_quant_type="nf4",
   bnb_4bit_use_double_quant=True,
   bnb_4bit_compute_dtype=torch.bfloat16
)

# 从指定模型名称或路径加载预训练语言模型
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    cache_dir=cache_dir,
    quantization_config=nf4_config,
    low_cpu_mem_usage=True
)

# 创建 tokenizer 并设置结束符号 (eos_token)
logging.getLogger('transformers').setLevel(logging.ERROR)
tokenizer = AutoTokenizer.from_pretrained(
    model_name,
    add_eos_token=True,
    cache_dir=cache_dir
)
tokenizer.pad_token = tokenizer.eos_token
```

```python
# 设置模型推理时的解码参数
max_len = 128
generation_config = GenerationConfig(
    do_sample=True,
    temperature=0.1,
    num_beams=1,
    top_p=0.3,
    no_repeat_ngram_size=3,
    pad_token_id=2,
)
```

主要函数解释:

- **BitsAndBytesConfig**: 配置模型的量化设置，使用 4 位精度以节省显存。
- **AutoModelForCausalLM**: 加载预训练的语言模型。
- **AutoTokenizer**: 加载对应的分词器。
- **GenerationConfig**: 设置文本生成时的参数，如温度、采样策略等。

这个代码块会下载对应的预训练模型，并加载到显存。

为什么要这样做

1. 模型量化，对于7B模型，通常加载到GPU中会占用至少14G显存，对于一些轻量级GPU很可能会报OOM(out of Memory)，因此需要通过量化技术来进行模型压缩，使得保持模型具有较好性能的同时降低显存消耗。相关内容可以参考 huggingface 的文档 [Bitsandbytes - Hugging Face 文档]
2. tokenizer 和 相关的 model 参数会在下载模型的时候一起下载，因此需要指定放置位置，对于 tokenizer，由于之前做了填充，需要告诉 tokenizer 之前做 padding 用的是什么token
3. GenerationConfig 里面有以下内容:
   a. `do_sample=True` 表示采样解码，False 表示贪婪解码
   b. `top_p` 保留累计概率p的token，0.3表示只考虑前30%概率的token
   c. `num_beams` 束搜索，>1时进行束搜索，寻找更优序列

初始表现

```python
""" 样例和 Prompt 都保持繁体 """

# 测试样例
```

```python
test_tang_list = [
    '相見時難別亦難，東風無力百花殘。',
    '重帷深下莫愁堂，臥後清宵細細長。',
    '芳辰追逸趣，禁苑信多奇。'
]

# 获取每个样例的模型输出
demo_before_finetune = []
for tang in test_tang_list:
    demo_before_finetune.append(
        f'模型輸入:\n以下是一首唐詩的第一句話，請用你的知識判斷並完成整首詩。{tang}\n\n模型輸出:\n' +
        evaluate('以下是一首唐詩的第一句話，請用你的知識判斷並完成整首詩。', generation_config, max_len, tang, verbose=False)
    )

# 打印并将输出存储到文本文件
for idx in range(len(demo_before_finetune)):
    print(f"Example {idx + 1}:")
    print(demo_before_finetune[idx])
    print("-" * 80)
```

```
Example 1:
模型輸入:
以下是一首唐詩的第一句話，請用你的知識判斷並完成整首詩。相見時難別亦難，東風無力百花殘。

模型輸出:
相會難，別亦艱難，東風吹，百花落。
--------------------------------------------------------------------------------
Example 2:
模型輸入:
以下是一首唐詩的第一句話，請用你的知識判斷並完成整首詩。重帷深下莫愁堂，臥後清宵細細長。

模型輸出:
重帷下，深下重帷，愁事重重，無計可施。
莫愁，莫愁莫愁。
堂，堂堂堂。
臥後，臥臥臥，清宵，清夢。
細，細長長。
--------------------------------------------------------------------------------
Example 3:
模型輸入:
以下是一首唐詩的第一句話，請用你的知識判斷並完成整首詩。芳辰追逸趣，禁苑信多奇。

模型輸出:
芳辰追逐趣，禁止苑中奇。
--------------------------------------------------------------------------------
```

可以看到，效果很差，很多不符合唐诗格式，且存在复读机。

设置用于微调的超参数

```
""" 强烈建议你尝试调整这个参数 """

num_train_data = 1040  # 设置用于训练的数据量，最大值为5000。通常，训练数据越多越好，模型会见到更多样化的诗句，从而提高生成质量，但也会增加训练时间。
                       # 使用默认参数(1040)：微调大约需要25分钟，完整运行所有单元大约需要50分钟。
                       # 使用最大值(5000)：微调大约需要100分钟，完整运行所有单元大约需要120分钟。

""" 你可以（但不一定需要）更改这些超参数 """

output_dir = "./output"  # 设置作业结果输出目录。
ckpt_dir = "./exp1"  # 设置 model checkpoint 保存目录（如果想将 model checkpoints 保存到其他目录下，可以修改这里）。
num_epoch = 1  # 设置训练的总 Epoch 数（数值越高，训练时间越长，若使用免费版的 Colab 需要注意时间太长可能会断线，本地运行不需要担心）。
LEARNING_RATE = 3e-4  # 设置学习率

""" 建议不要更改此单元格中的代码 """

cache_dir = "./cache"  # 设置缓存目录路径
from_ckpt = False  # 是否从 checkpoint 加载模型权重，默认值为否
ckpt_name = None  # 加载特定 checkpoint 时使用的文件名，默认值为无
dataset_dir = "./GenAI-Hw5/Tang_training_data.json"  # 设置数据集目录或文件路径
logging_steps = 20  # 定义训练过程中每隔多少步骤输出一次日志
save_steps = 65  # 定义训练过程中每隔多少步骤保存一次模型
save_total_limit = 3  # 控制最多保留多少个模型 checkpoint
report_to = "none"  # 设置不上报实验指标，也可以设置为 "wandb"，此时需要获取对应的 API，见：https://github.com/Hoper-J/AI-Guide-and-Demos-zh_CN/pull/5
MICRO_BATCH_SIZE = 4  # 定义微批次大小
BATCH_SIZE = 16  # 定义一个批次的大小
GRADIENT_ACCUMULATION_STEPS = BATCH_SIZE // MICRO_BATCH_SIZE  # 计算每个微批次累积的梯度步骤
CUTOFF_LEN = 256  # 设置文本截断的最大长度
```

```python
LORA_R = 8  # 设置 LORA（Layer-wise Random Attention）的 R 值
LORA_ALPHA = 16  # 设置 LORA 的 Alpha 值
LORA_DROPOUT = 0.05  # 设置 LORA 的 Dropout 率
VAL_SET_SIZE = 0  # 设置验证集的大小，默认值为无
TARGET_MODULES = ["q_proj", "up_proj", "o_proj", "k_proj", "down_proj", "gate_proj", "v_proj"]  # 设置目标模块，这些模块的权重将被保存为 checkpoint。
device_map = "auto"  # 设置设备映射，默认值为 "auto"
world_size = int(os.environ.get("WORLD_SIZE", 1))  # 获取环境变量 "WORLD_SIZE" 的值，若未设置则默认为 1
ddp = world_size != 1  # 根据 world_size 判断是否使用分布式数据处理（DDP），若 world_size 为 1 则不使用 DDP
if ddp:
    device_map = {"": int(os.environ.get("LOCAL_RANK") or 0)}
    GRADIENT_ACCUMULATION_STEPS = GRADIENT_ACCUMULATION_STEPS // world_size
```

Note

参数 `report_to` 设置为 `None` 是无效的，正确的做法是设置为 `"none"` [^1]。

这部分参数都设置了些什么？

1. **训练数据配置**：设置训练样本数量仅为 1040 条，不过原始训练数据内包含5000条数据，也可以均用于训练
2. **LoRA**：通过一个小的秩矩阵进行参数微调，`LORA_R=8` 表示矩阵秩=8，常用4，8，16.[一文吃透 LORA：从原理到应用，大模型轻量微调技术全解析_lora微调原理-CSDN博客]
3. `LEARNING_RATE` 和 `num_epochs` 通常设置都会比较小，因为SFT阶段我们通常只是想要微调

开始微调

```python
# 设置TOKENIZERS_PARALLELISM为false，这里简单禁用并行性以避免报错
os.environ["TOKENIZERS_PARALLELISM"] = "false"
```

```
# 创建指定的输出目录
os.makedirs(output_dir, exist_ok=True)
os.makedirs(ckpt_dir, exist_ok=True)

# 根据 from_ckpt 标志，从 checkpoint 加载模型权重
if from_ckpt:
    model = PeftModel.from_pretrained(model, ckpt_name)

# 对量化模型进行预处理以进行训练
model = prepare_model_for_kbit_training(model)

# 使用 LoraConfig 配置 LORA 模型
config = LoraConfig(
    r=LORA_R,
    lora_alpha=LORA_ALPHA,
    target_modules=TARGET_MODULES,
    lora_dropout=LORA_DROPOUT,
    bias="none",
    task_type="CAUSAL_LM",
)
model = get_peft_model(model, config)

# 将 tokenizer 的填充 token 设置为 0
tokenizer.pad_token_id = 0

# 加载并处理训练数据
with open(dataset_dir, "r", encoding="utf-8") as f:
    data_json = json.load(f)
with open("tmp_dataset.json", "w", encoding="utf-8") as f:
    json.dump(data_json[:num_train_data], f, indent=2,
ensure_ascii=False)

data = load_dataset('json', data_files="tmp_dataset.json",
download_mode="force_redownload")

# 将训练数据分为训练集和验证集（若 VAL_SET_SIZE 大于 0）
if VAL_SET_SIZE > 0:
    train_val = data["train"].train_test_split(
        test_size=VAL_SET_SIZE, shuffle=True, seed=42
    )
    train_data =
train_val["train"].shuffle().map(generate_training_data)
    val_data =
```

```
train_val["test"].shuffle().map(generate_training_data)
else:
    train_data =
data['train'].shuffle().map(generate_training_data)
    val_data = None

# 使用 Transformers Trainer 进行模型训练
trainer = transformers.Trainer(
    model=model,
    train_dataset=train_data,
    eval_dataset=val_data,
    args=transformers.TrainingArguments(
        per_device_train_batch_size=MICRO_BATCH_SIZE,
        gradient_accumulation_steps=GRADIENT_ACCUMULATION_STEPS,
        warmup_steps=50,
        num_train_epochs=num_epoch,
        learning_rate=LEARNING_RATE,
        fp16=True,  # 使用混合精度训练
        logging_steps=logging_steps,
        save_strategy="steps",
        save_steps=save_steps,
        output_dir=ckpt_dir,
        save_total_limit=save_total_limit,
        ddp_find_unused_parameters=False if ddp else None,  # 是否
使用 DDP，控制梯度更新策略
        report_to=report_to,
    ),

data_collator=transformers.DataCollatorForLanguageModeling(tokeniz
er, mlm=False),
)

# 禁用模型的缓存功能
model.config.use_cache = False

# 若使用 PyTorch 2.0 以上版本且非 Windows 系统，编译模型
if torch.__version__ >= "2" and sys.platform != 'win32':
    model = torch.compile(model)

# 开始模型训练
trainer.train()

# 将训练好的模型保存到指定目录
```

```
model.save_pretrained(ckpt_dir)

# 打印训练过程中可能出现的缺失权重警告信息
print("\n 如果上方有关于缺少键的警告，请忽略 :)")
```

训练完后，会在 `exp1` 文件夹中生成 `ckpt` 文件，这个就是微调后的参数，但是只有这个参数是不能直接运行的，我们还要结合原本的参数。

测试微调后的模型

选取要加载的微调后的模型

```
# 查找所有可用的 checkpoints
ckpts = []
for ckpt in os.listdir(ckpt_dir):
    if ckpt.startswith("checkpoint-"):
        ckpts.append(ckpt)

# 列出所有的 checkpoints
ckpts = sorted(ckpts, key=lambda ckpt: int(ckpt.split("-")[-1]))
print("所有可用的 checkpoints: ")
print(" id: checkpoint 名称")
for (i, ckpt) in enumerate(ckpts):
    print(f"{i:>3}: {ckpt}")
```

```
""" 你可以（但不一定需要）更改 checkpoint """

id_of_ckpt_to_use = -1  # 要用于推理的 checkpoint 的 id（对应上一单元格的输出结果）。
                        # 默认值 -1 表示使用列出的最后一个 checkpoint。
                        # 如果你想选择其他 checkpoint，可以将 -1 更改为列出的 checkpoint id 中的某一个。

ckpt_name = os.path.join(ckpt_dir, ckpts[id_of_ckpt_to_use])

""" 你可以（但不一定需要）更改解码参数 """
```

```
# 你可以在此处调整解码参数，解码参数的详细解释请见作业幻灯片。
max_len = 128  # 生成回复的最大长度
temperature = 0.1  # 设置生成回复的随机度，值越小生成的回复越稳定。
top_p = 0.3  # Top-p (nucleus) 采样的概率阈值，用于控制生成回复的多样性。
# top_k = 5  # 调整 Top-k 值，以增加生成回复的多样性并避免生成重复的词汇。
```

释放显存

为防止显存不足，我们先释放之前占用的显存。

```
import gc

# 删除训练相关的所有对象
if 'trainer' in locals():
  del trainer
if 'train_data' in locals():
  del train_data
if 'val_data' in locals() and val_data is not None:
  del val_data
if 'data' in locals():
  del data

# 删除模型和 tokenizer 对象
del model
del tokenizer

# 调用垃圾回收机制，强制释放未使用的内存
gc.collect()

# 清理 GPU 缓存
torch.cuda.empty_cache()
```

加载模型和分词器

```
test_data_path = "GenAI-Hw5/Tang_testing_data.json"  # 测试数据集的路径
output_path = os.path.join(output_dir, "results.txt")  # 生成结果的
```

```
输出路径

cache_dir = "./cache"
seed = 42
no_repeat_ngram_size = 3

# 配置模型的量化设置
nf4_config = BitsAndBytesConfig(
   load_in_4bit=True,
   bnb_4bit_quant_type="nf4",
   bnb_4bit_use_double_quant=True,
   bnb_4bit_compute_dtype=torch.bfloat16
)

# 加载 tokenizer
tokenizer = AutoTokenizer.from_pretrained(
    model_name,
    cache_dir=cache_dir,
    quantization_config=nf4_config
)

# 加载模型
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=nf4_config,
    device_map={'': 0},
    cache_dir=cache_dir
)

# 加载微调后的权重
model = PeftModel.from_pretrained(model, ckpt_name, device_map=
{'': 0})
```

生成测试结果

```
""" 建议不要更改此单元格中的代码 """

results = []

# 设置生成配置，包括随机度、束搜索等参数
generation_config = GenerationConfig(
```

```
    do_sample=True,
    temperature=temperature,
    num_beams=1,
    top_p=top_p,
    # top_k=top_k,  # 如果需要使用 top-k，可以在此设置
    no_repeat_ngram_size=no_repeat_ngram_size,
    pad_token_id=2
)

# 读取测试数据集
with open(test_data_path, "r", encoding="utf-8") as f:
    test_datas = json.load(f)

# 对每个测试样例生成预测，并保存结果
with open(output_path, "w", encoding="utf-8") as f:
    for (i, test_data) in enumerate(test_datas):
        predict = evaluate(test_data["instruction"],
generation_config, max_len, test_data["input"], verbose=False)
        f.write(f"{i+1}. " + test_data["input"] + predict + "\n")
        print(f"{i+1}. " + test_data["input"] + predict)
```

```
1. 雪霽銀妝素，桔高映瓊枝。春色未開時，花落不聞聲。
2. 夫子何爲者？栖栖一代中。
3. 飛蓋去芳園，蘭橈遊翠渚。春色未盡，花落未央。
4. 條風開獻節，灰律動初陽。春色正未發，春聲已先通。
5. 昨夜星辰昨夜風，畫樓西畔桂堂東。春色未到春色遠，春色到時春色空。
6. 三日入廚下，洗手作羹湯。爲君作酒食，不爬不牕。
7. 嵩雲秦樹久離居，雙鯉迢迢一紙書。千載不聞胡歌聲，一葉獨在江邊秋。
8. 慨然撫長劒，濟世豈邀名。今朝不復見，何須留情情。
9. 乘興南遊不戒嚴，九重誰省諫書函。春色千載不相忘，秋雨萬載不復見。
10. 猿鳥猶疑畏簡書，風雲常爲護儲胥。再爷爵不待言，燼燼恩已昭昭。
11. 君問歸期未有期，巴山夜雨漲秋池。長門無路入城去，孤燈獨照空城樓。
12. 相見時難別亦難，東風無力百花殘。春色已去春意也，春意已去春風也。
13. 雲母屏風燭影深，長河漸落曉星沈。玉樓秋雨夜不眠，玉樓春雨春不眠。
14. 高閣客竟去，小園花亂飛。春色無依歸，春意無依依。
15. 瑤池阿母綺窗開，黃竹歌聲動地哀。春色千載不相見，今朝重來又何在？
```

会发现虽然会有些乱码，但是效果上比先前的好了不少。

微调前后的对比

```python
# 使用之前的测试例子
test_tang_list = [
    '相見時難別亦難，東風無力百花殘。',
    '重帷深下莫愁堂，臥後清宵細細長。',
    '芳辰追逸趣，禁苑信多奇。'
]

# 使用微调后的模型进行推理
demo_after_finetune = []
for tang in test_tang_list:
    demo_after_finetune.append(
        f'模型輸入:\n以下是一首唐詩的第一句話，請用你的知識判斷並完成整首詩。{tang}\n\n模型輸出:\n' +
        evaluate('以下是一首唐詩的第一句話，請用你的知識判斷並完成整首詩。', generation_config, max_len, tang, verbose=False)
    )

# 打印输出结果
for idx in range(len(demo_after_finetune)):
    print(f"Example {idx + 1}:")
    print(demo_after_finetune[idx])
    print("-" * 80)
```

```
Example 1:
模型輸入:
以下是一首唐詩的第一句話，請用你的知識判斷並完成整首詩。相見時難別亦難，東風無力百花殘。

模型輸出:
春色已去春意也，春意已去春風也。
--------------------------------------------------------------------------------
Example 2:
模型輸入:
以下是一首唐詩的第一句話，請用你的知識判斷並完成整首詩。重帷深下莫愁堂，臥後清宵細細長。

模型輸出:
月色入簾照玉床，風聲入耳吹金管。
--------------------------------------------------------------------------------
```

可以看到和最开始有着很大的差别，最起码不是复读机了。

MDD5k数据集介绍

MDD-5k是由盛大集团等机构合作开发的中文精神障碍诊断对话数据集，基于1000个经过清洗的真实患者案例，包含5000个高质量的诊断对话，每个对话平均26.8轮，共6906.8个中文字符。数据集通过神经符号多代理框架生成，确保了对话的多样性和专业性，主要用于辅助诊断聊天机器人和精神障碍分类等下游任务，旨在解决精神健康领域的诊断难题。

[lemonsis/MDD-5k: Official Implementation of AAAI 2025 paper "MDD-5k: A New Diagnostic Conversation Dataset for Mental Disorders Synthesized via Neuro-Symbolic LLM Agents"]

数据集基本格式示例，对于一个patient：

```
[
    {
        "conversation": [
            {
                "doctor": "你好，我是今天负责你问诊的医生。你可以随便叫我张医生。能跟我聊聊最近你的心情怎么样吗？有没有什么特别困扰你的事情？",
                "patient": "最近我一直觉得情绪很低落，做什么都提不起兴趣。晚上睡觉特别困难，经常半夜醒来，还做很多噩梦。两个月前这些症状更严重了，总是感觉烦躁，对家人发脾气。我胃口不好，体重也减轻了不少。其实这不是第一次有这种情况，之前在新纪元医院看过，被诊断为中度抑郁，当时吃了舍曲林和奥平片，但效果不理想，这次病情又加重了。我经常觉得活着没意思，有时候甚至有自残和自杀的想法。学习上也很难集中注意力，总是感觉疲劳和精力不足，头晕眼花的情况也不少见。生活上的情感问题和学习压力让我感觉特别迷茫。这一切真的让我非常困扰，希望能找到有效的方法改善这种状态。"
            },
            {
                "doctor": "能跟我详细讲讲你现在的学习情况吗？比如你每天学习多久，最近有没有觉得比以前更难集中注意力或者完成任务？这些症状对你的学业有多大影响呢？",
                "patient": "最近学习情况真的不太好，每天大概只能集中注意力一两个小时，很多时候书本摊开了，但脑子却一片空白。比以前难多了，完成任务也很吃力。总感觉思维变慢，常常要看好多遍才能理解。我现在高一的课业本来就重，这样下来真的很难跟上进度。每次老师提问我都紧张得心跳加速，很害怕答不上来，更别提考试了，成绩直线下滑。这种状况让我非常焦虑，有时甚至连教室都不想进去，只想逃避一切。"
            },
            ...
        ]
    }
]
```

每一轮对话均为一个 doctor 与一个 patient 展开对话，我们的目标是根据这些数据，微调一个 doctor 模型。

任务建议

- 模型：建议使用 Qwen3-4B 或者 Qwen3-1.7B 完成
- 数据：可以尝试使用30个 `patient.json` 的数据用于微调，用 1 个 `patient.json` 的数据用于评测
- Metric：可以简单使用 ROUGE 或者 BLEU 进行评测，计算测试集上每轮对话里面原始的doctor回答与微调后模型回答的一致程度。
- 训练方式：可以尝试修改提供的代码，也可以尝试使用 LlamaFactory 进行微调(注意**数据格式**需要进行调整)
  - 第二种方法相对较为简单

其他参考内容

- [[特殊字符] Qwen 多轮对话微调数据集格式全解析：从零构建高质量训练数据-CSDN博客]
- [多轮对话数据微调 qwen | Sharpen's Blogs]
- [2.3 llama factory微调Qwen3 微调&评估&导出_哔哩哔哩_bilibili]
- [【微调03】零代码！训练、推理、合并、部署、调用，一个视频体验微调全流程_哔哩哔哩_bilibili]
- [基于LLaMA-Factory框架微调多轮对话LLM - 知乎]
- [LLaMA-Factory：手把手教你从零微调大模型！ - 编程大咖]